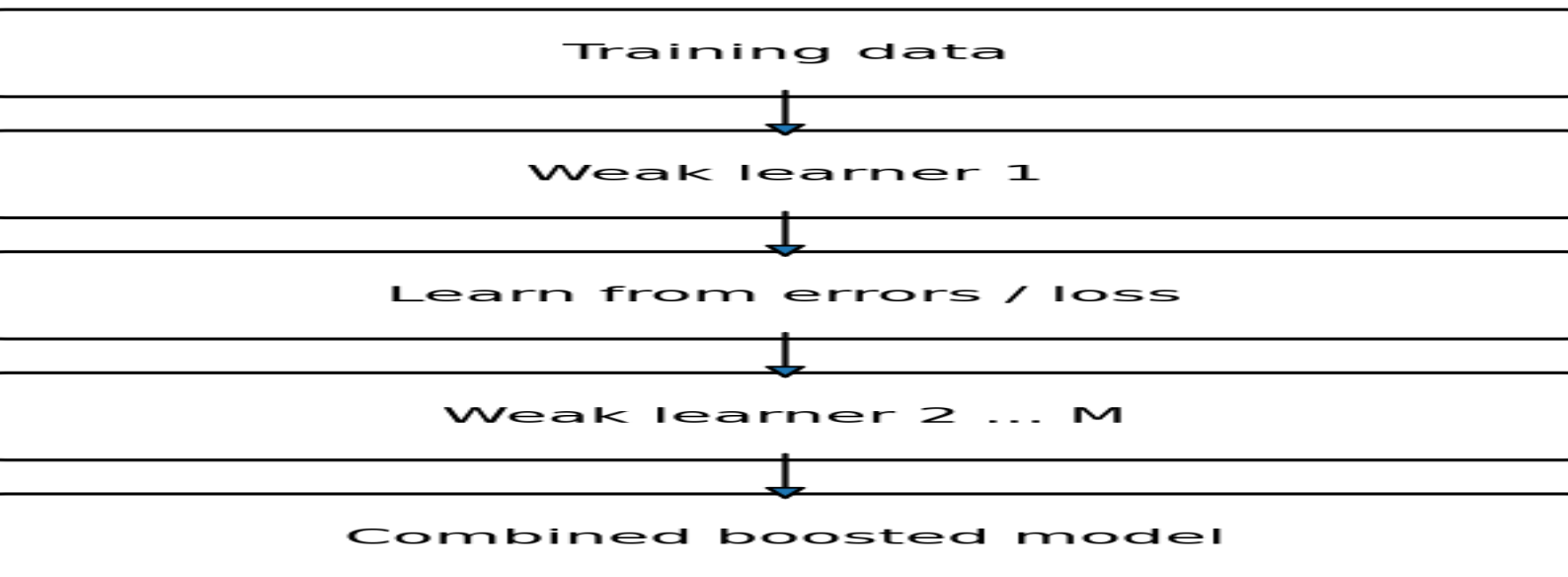


DAY 29

Boosting Algorithms in Machine Learning

Detailed guide with diagrams, concepts, complete Python project, code walkthrough, evaluation, and practice solutions



60 Days of Data Science Learning Journey

Prepared for Muhammad Ahmad

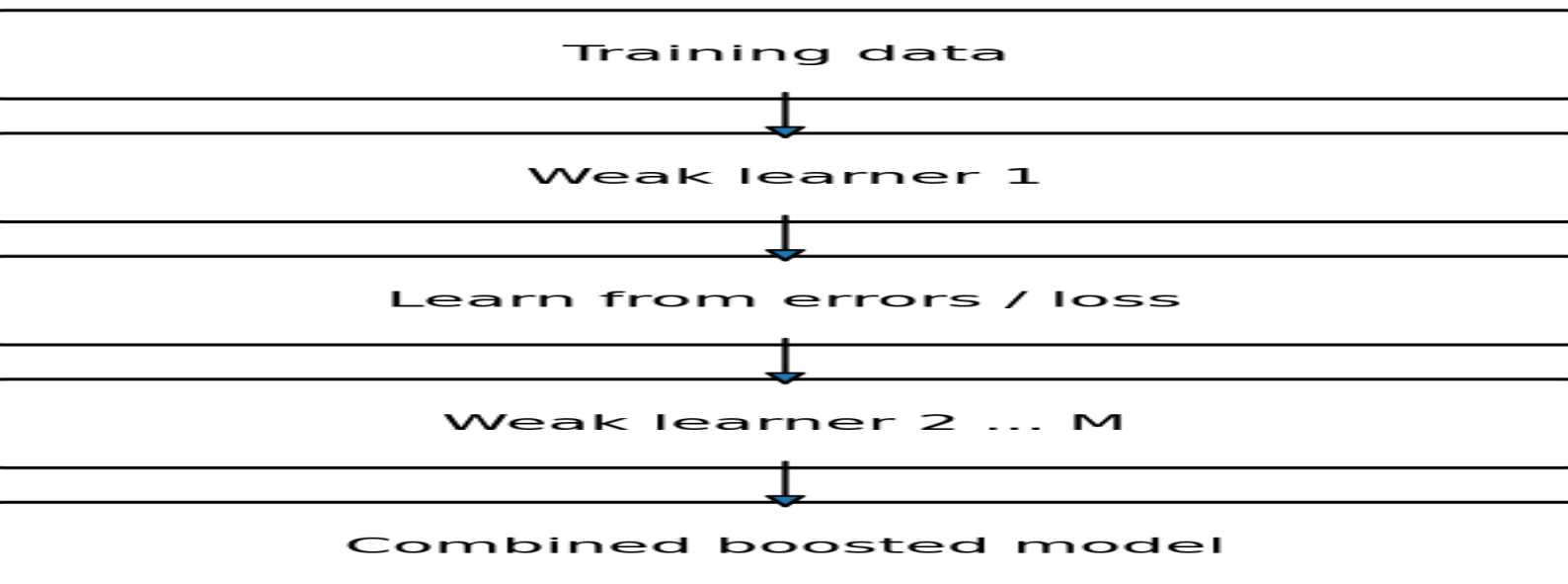
1. Learning objectives

- Define ensemble learning and weak learners.
- Explain why Boosting is sequential.
- Describe classic AdaBoost sample reweighting.
- Explain Gradient Boosting and loss optimization.
- Compare Random Forest, AdaBoost, and Gradient Boosting.
- Train and evaluate a classifier with Scikit-learn.

2. Ensemble learning and weak learners

Ensemble learning combines predictions from several models to improve predictive performance or stability. A weak learner is a model that performs only modestly better than a simple baseline on the task. Shallow Decision Trees are commonly used as weak learners.

Boosting trains learners sequentially. Each new learner is influenced by the current ensemble's weaknesses. AdaBoost and Gradient Boosting use different mechanisms, so they should not be treated as identical algorithms.



Remember: Boosting does not guarantee better performance on every dataset. Noisy labels, poor features, unsuitable settings, and excessive model complexity can hurt results.

3. AdaBoost: Adaptive Boosting

In classic AdaBoost, training examples start with weights. A weak learner is fitted using those weights. Misclassified examples receive more relative weight, so later learners focus more on examples the current model struggles with. The final ensemble combines learners using weights related to their performance.

A decision stump is a Decision Tree with `max_depth=1`. It is a common simple learner for AdaBoost.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

weak_learner = DecisionTreeClassifier(
    max_depth=1,
    random_state=42
)

ada_model = AdaBoostClassifier()
```

```

estimator=weak_learner,
n_estimators=50,
learning_rate=1.0,
random_state=42
)

```

- **estimator**: base learner repeated across boosting stages.
- **n_estimators**: maximum number of stages.
- **learning_rate**: scales each learner's contribution. Smaller values may require more stages.
- **random_state**: supports reproducibility where randomness is involved.

Classic AdaBoost may be sensitive to outliers and mislabeled examples because hard-to-classify samples can gain substantial weight.

4. Gradient Boosting

Gradient Boosting builds an additive model one stage at a time. Each new learner aims to reduce a chosen loss function. In squared-error regression, a useful intuition is that each stage learns from residuals. For classification and other losses, the algorithm uses gradients of the loss, so it is not always just fitting raw residuals.

Residual in a basic regression example = Actual value – Predicted value.

Actual marks	Predicted marks	Residual
80	70	10
65	72	-7
90	85	5

A positive residual means underprediction in this example. A negative residual means overprediction. The next learner is used to improve the current ensemble.

```

from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)

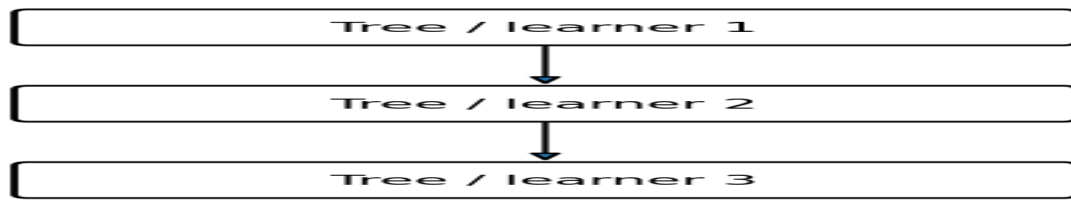
```

- **n_estimators**: number of boosting stages.
- **learning_rate**: contribution of each stage.
- **max_depth**: complexity of the individual trees.
- **random_state**: helps make results repeatable.

A smaller learning rate often requires more stages. More stages or deeper trees can improve or harm results depending on the data. Use validation to choose settings.

5. Random Forest vs. Boosting

AdaBoost



Reweight difficult examples
(classic AdaBoost)

Feature	Random Forest	AdaBoost	Gradient Boosting
Family	Bagging	Boosting	Boosting
Training	Trees can train independently	Sequential	Sequential
Mechanism	Bootstrap samples and random features	Classic version reweights difficult samples	Learners reduce loss stage by stage
Combination	Vote or average	Weighted combination	Additive combination
Common concern	May overfit in some settings	Can focus on noisy examples	Excess complexity may overfit

Memory trick: Random Forest combines randomized trees. Boosting builds a sequence of learners, with later stages guided by errors or loss.

6. Complete project: Student Pass/Fail Prediction

This example predicts a binary label using study hours, attendance, and previous marks. The data are synthetic and small. They are for practicing a workflow, not for real educational decisions.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

data = {
    "study_hours": [
        1, 2, 3, 4, 5, 6, 7, 8,
        2, 3, 4, 6, 7, 8, 9, 10,
        1, 3, 5, 7, 9, 2, 6, 10
    ],
    "attendance": [
        40, 45, 50, 55, 60, 65, 70, 80,
        50, 55, 60, 70, 75, 85, 90, 95,
        35, 48, 68, 72, 88, 42, 78, 98
    ],
    "previous_marks": [
        30, 35, 40, 45, 50, 55, 60, 75,
        38, 42, 48, 58, 65, 80, 85, 90,
        25, 45, 55, 68, 88, 32, 72, 95
    ],
    "passed": [
        0, 0, 0, 0, 1, 1, 1, 1,
        0, 0, 0, 1, 1, 1, 1, 1,
        0, 0, 1, 1, 1, 0, 1, 1
    ]
}

df = pd.DataFrame(data)

X = df[["study_hours", "attendance", "previous_marks"]]
y = df["passed"]

X_train, X_test, y_train, y_test = train_test_split(
```

```

    X, y, test_size=0.25, random_state=42, stratify=y
)

model = GradientBoostingClassifier(
    n_estimators=50,
    learning_rate=0.1,
    max_depth=2,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification report:")
print(classification_report(y_test, y_pred, zero_division=0))

new_student = pd.DataFrame([
    "study_hours": 6,
    "attendance": 80,
    "previous_marks": 70
])

prediction = model.predict(new_student)[0]
print("Prediction:", "Likely to pass" if prediction == 1
      else "At risk of failing")

```

7. Code walkthrough

Imports: Pandas builds the table. `train_test_split` separates data. `GradientBoostingClassifier` builds the model. Metrics functions evaluate results.

DataFrame: The dictionary stores columns. Every list must have the same number of entries.

Features and target: X contains the three inputs. y contains the passed label, with 1 for pass and 0 for not pass.

Train/test split: 75% of the data is used for training and 25% for testing. `stratify=y` attempts to preserve class proportions. `random_state` makes the split reproducible.

Model settings: The model uses 50 stages, `learning_rate` 0.1, and tree depth 2. These are example settings, not guaranteed optimal settings.

Training: `model.fit` learns patterns from the training features and labels.

Prediction: `model.predict` generates labels for test examples that were not used for fitting.

Evaluation: Accuracy, confusion matrix, and classification report provide different views of performance.

New input: The new student's DataFrame uses the same feature names as training data. The prediction is not a guarantee about a real person.

8. Classification metrics

- **Accuracy:** fraction of all predictions that are correct. It can be misleading on imbalanced data.
- **Precision:** among predictions labeled positive, the fraction that truly are positive.
- **Recall:** among actual positive examples, the fraction correctly found.
- **F1-score:** harmonic mean of precision and recall.
- **Confusion matrix:** shows correct and incorrect predictions. For binary labels ordered 0 then 1, Scikit-learn typically displays `[[TN, FP], [FN, TP]]`.

Do not assume a fixed accuracy before running the code. Results depend on the data, split, model settings, and software version.

9. Practice tasks with solution code

Task A: Train AdaBoost on the same split

```
from sklearn.ensemble import AdaBoostClassifier
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=50,
    learning_rate=0.5,
    random_state=42
)
ada.fit(X_train, y_train)
ada_pred = ada.predict(X_test)
print("AdaBoost accuracy:", accuracy_score(y_test, ada_pred))
print(classification_report(y_test, ada_pred, zero_division=0))

```

Explanation: this reuses the exact same split, which makes a basic comparison more meaningful. On a very small dataset, results are still unstable.

Task B: Compare learning rates

```

for rate in [0.05, 0.1, 0.2]:
    candidate = GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=rate,
        max_depth=2,
        random_state=42
    )
    candidate.fit(X_train, y_train)
    pred = candidate.predict(X_test)
    print(rate, accuracy_score(y_test, pred))

```

For rigorous tuning, compare settings with cross-validation on the training data and reserve the test set for final evaluation.

Task C: Compare Random Forest

```

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
print("Random Forest accuracy:", accuracy_score(y_test, rf_pred))

```

Compare precision, recall, F1-score, and confusion matrices as well as accuracy.

Task D: Why can accuracy mislead?

If 95% of examples are in one class, a model that always predicts that class can get 95% accuracy while failing to identify the minority class. Always inspect class balance and suitable metrics.

10. Common mistakes and best practices

- Assuming all boosting algorithms work in the same way.
- Assuming more estimators always improve the model.
- Tuning repeatedly on the test set, which biases the final estimate.
- Ignoring noisy labels or outliers, especially for classic AdaBoost.
- Reporting accuracy without checking precision, recall, F1-score, or class balance.
- Treating synthetic toy data as evidence of real-world performance.
- Allowing target-derived or future information into features, causing data leakage.
- Treating feature importance or model output as proof of cause or certainty.

11. Revision checklist

- ■ I can explain ensemble learning and weak learners.
- ■ I can explain classic AdaBoost sample reweighting.
- ■ I can explain Gradient Boosting as loss optimization.
- ■ I can compare Random Forest with boosting algorithms.

- ■ I can train and evaluate a Scikit-learn classifier.
- ■ I understand accuracy, precision, recall, F1-score, and confusion matrices.
- ■ I understand overfitting, data leakage, and the limits of tiny datasets.

12. Final summary

Boosting combines weak learners sequentially. Classic AdaBoost adjusts the relative importance of training examples, while Gradient Boosting adds learners to reduce a chosen loss function. Random Forest combines randomized trees that can generally be trained independently. No algorithm wins on every dataset, so use sound validation, appropriate metrics, and a held-out test set.

Next topic suggestion: Day 30 can introduce XGBoost and other modern gradient-boosting methods, including tuning and practical use cases.